



9.2 性能调优

性能调优总体思路

- 总体思路

- openGauss数据库的总体性能调优思路为：分析系统硬件资源瓶颈和SQL瓶颈，对性能瓶颈作针对性调优，使数据库性能达到可接受的范围。
- openGauss性能调优过程需要综合考虑多方面因素，因此，调优人员应对数据库、存储系统、网络、操作系统甚至服务器内核架构等有广泛而深刻的理解。
- 性能调优过程有时需要重启openGauss，可能会中断当前业务。因此，业务上线后，当性能调优操作需要重启openGauss时，操作窗口时间需向管理部门提出申请，经批准后方可执行。

影响性能的因素

- **硬件资源**

- CPU、内存、磁盘、网络等。

- **软件资源**

- 操作系统，包含其参数配置、驱动程序等，数据库软件等。

- **负载与系统规模**

- 数据量、并发量、最大吞吐量等。

- **数据库设计**

- 数据库架构设计、表结构设计、索引设计、查询设计、数据库参数配置等。

调优流程 – 分析系统性能

- (1) **CPU**：使用top等命令查看CPU利用率是否过高，一般整体CPU利用率超过80%即过高。
- (2) **内存**：使用free等命令查看内存的占用情况，是否可用物理内存已经被占满，是否物理内存过小。
- (3) **磁盘I/O**：使用iostat等命令查看磁盘占用情况，是否磁盘利用率超过90%，如果磁盘成为瓶颈，利用率会在90%以上。如果SQL中有大量的Insert或者Update，需要重点关注是否将磁盘打满。
- (4) **网络**：使用ping、sar等命令查看网络时延、带宽是否存在瓶颈，好的网络时延应小于0.1ms，带宽利用率小于10%。

调优流程 – 分析基本的参数配置 (1)

- 在正式分析SQL慢之前，首先需要关注一些重要的GUC参数配置，往往性能的劣化都跟参数设置不合理相关。

参数名称	参数含义	最佳实践
max_process_memory	设置一个数据库节点可用的最大物理内存。	根据物理内存大小调整： $(\text{物理内存大小} - \text{vm.min_free_kbytes}) * 0.7$ 即：max_process_memory = 物理内存 * 0.665
shared_buffers	设置openGauss使用的共享内存大小。	建议设置shared_buffers值为内存的40%以内，行存列存分开对待，行存设大，列存设小。列存： $(\text{单服务器内存} / \text{单服务器数据库节点个数}) * 0.4 * 0.25$ 。
work_mem	设置内部排序操作和Hash表在开始写入临时磁盘文件之前使用的内存大小。	依据查询特点和并发来确定，一旦work_mem限定的物理内存不够，算子运算数据将写入临时表空间，带来5-10倍的性能下降，查询响应时间从秒级下降到分钟级。

调优流程 – 分析基本的参数配置 (2)

- `max_process_memory`

- 该系数的目的是尽可能保证系统的可靠性，不会因数据库内存膨胀导致节点OOM。这个公式中提到 `vm.min_free_kbytes`，其含义是预留操作系统内存供内核使用，通常用作操作系统内核中通信收发内存分配，至少为5%内存。
- 当该值设置不合理，即大于服务器物理内存，可能导致操作系统OOM问题。
- 若一台机器上部署多个数据库实例时，需要缩小相关配置。

调优流程 – 分析基本的参数配置 (3)

- shared_buffers

- shared_buffers需要设置为BLCKSZ的整数倍，BLCKSZ目前设置为8kB，即shared_buffers需要设置为8kB整数倍。改变BLCKSZ的值会改变最小值。
- 如果设置较大的shared_buffers需要同时增加checkpoint_segments的值，因为写入大量新增、修改数据需要消耗更多的时间周期。
- 在资源池化模式下，如果主备机同时运行业务，应适当增大本参数，避免该参数设置过小导致主备机页面频繁交互，增大网络负载，严重情况下，可能出现数据库响应较慢甚至卡住的情况。

调优流程 – 分析基本的参数配置 (4)

- work_mem

- 对于串行无并发的复杂查询场景，平均每个查询有5-10关联操作，建议work_mem=50%内存/10。
- 对于串行无并发的简单查询场景，平均每个查询有2-5个关联操作，建议work_mem=50%内存/5。
- 对于并发场景，建议work_mem=串行下的work_mem/物理并发数。
- 对于BitmapScan的哈希表也会受到work_mem的限制，但不会被严格管控下盘。完全Lossify的情况下，哈希表每占用1MB的内存，对应一次BitmapHeapScan的16GB的页面（Ustore为32GB），达到work_mem上限后，会按此比例随数据访问量线性增长。

调优流程 – 分析SQL性能

- 在检查基本的配置参数无异常的情况下，我们再深入分析SQL本身的性能问题。
 - (1) 收集SQL中涉及到的所有表的统计信息。
 - (2) 通过查看执行计划来查找原因。
 - (3) 审视和修改表定义。
 - (4) 针对EXPLAIN或EXPLAIN PERFORMANCE信息，查看实际行和估算行偏差，偏差大说明是统计信息的问题。
 - (5) 分析执行计划的谓词、索引等，查看是否正确创建索引，索引是否使用适当。
 - (6) 分析hash 索引、sort等有没有溢出，有溢出的话会在磁盘上产生临时文件。
 - (7) 如果执行计划没有什么优化空间，分析火焰图、冷焰图等。

查询最耗性能的SQL (1)

- 系统中有些SQL语句运行了很长时间还没有结束，这些语句会消耗很多的系统性能。
 - 1. 查询系统中长时间运行的查询语句。

```
SELECT current_timestamp - query_start AS runtime, datname, username, query FROM pg_stat_activity where state != 'idle' ORDER BY 1 desc;
```

- 查询后会按执行时间从长到短顺序返回查询语句列表，第一条结果就是当前系统中执行时间最长的查询语句。返回结果中包含了系统调用的SQL语句和用户执行SQL语句，请根据实际找到用户执行时间长的语句。
- 若当前系统较为繁忙，可以通过限制current_timestamp - query_start大于某一阈值来查看执行时间超过此阈值的查询语句。

```
SELECT query FROM pg_stat_activity WHERE current_timestamp - query_start > interval '1 days';
```

- 2. 设置参数track_activities为on。

```
SET track_activities = on;
```

- 当此参数为on时，数据库系统才会收集当前活动查询的运行信息。

查询最耗性能的SQL (2)

- 3. 查看正在运行的查询语句。以查看视图pg_stat_activity为例：

```
SELECT datname, username, state FROM pg_stat_activity;  
datname | username | state |  
-----+-----+-----+  
postgres | omm      | idle  |  
postgres | omm      | active|  
(2 rows)
```

- 4. 分析长时间运行的查询语句状态。
 - 若查询语句处于正常状态，则等待其执行完毕。
 - 若查询语句阻塞，则通过如下命令查看当前处于阻塞状态的查询语句：

本语句中包含了当前被阻塞的查询语句，该查询语句所请求的锁资源可能被其他会话持有，正在等待持有会话释放该资源。

```
SELECT datname, username, state, query FROM pg_stat_activity WHERE waiting = true;
```

- 注意：只有当查询阻塞在系统内部锁资源时，waiting字段才显示为true。尽管等待锁资源是数据库系统最常见的阻塞行为，但是在某些场景下查询也会阻塞在等待其他系统资源上，例如写文件、定时器等。但是这种情况的查询阻塞，不会在视图pg_stat_activity中体现。

目录

1. 性能调优的总体思路与流程
2. 数据库整体性能调优
3. **SQL调优**
 - 相似算子选择及Plan Hint
 - 索引
 - 表结构优化
 - 优化器CBO
 - SQL性能诊断
 - 子查询调优和SQL语句改写

索引优化原理

● 1、实现原理

- 索引作为一种高效查询的数据结构，其在openGauss中的实现结构通常为B+树。利用表的某些常用的查询列的数据构建树形结构，以空间换时间的手段，优化每次查询的性能。

● 2、查询流程

- （1）先扫描一遍索引，从索引中找到符合要求的记录的位置(指针)，再定位到表中具体的Page去获取对应的元组，即先走索引，再读表数据。
- （2）随着查询数据的条数增多，即访问索引树的次数增多，索引扫描不仅要扫描表数据，还需要扫描索引树，因此其性能会逐渐不如全表扫描。

● 3、关于创建索引的建议

- 选择率较高的WHERE条件和ORDER BY字段建立索引。在openGauss中，如果选择率低于30%左右，即返回结果集占有所有数据的占比超过70%时，优化器可能会选择全表扫描而不去选择索引扫描。

索引失效

- 1、什么是索引失效？
 - 索引失效是指在某种特定SQL语法或特定场景下，导致实际执行过程中不选择更优的索引扫描，而是选择性能更差的全表扫描。索引失效造成的影响对于大表更为明显。
- 2、哪种语法会导致索引失效？
 - 在openGauss单表查询语句中，会导致索引失效的语法包括：
 - （1）索引列使用函数
 - （2）索引列执行计算
 - （3）索引列发生隐式类型转换
 - （4）WHERE中的OR语句
 - （5）索引列使用模糊匹配条件等

索引失效 – 示例 (1)

- 创建表t1，第一列 c1和第三列c3 创建有索引。

```
create table t1(c1 int primary key, c2 int, c3 text unique);
```

- 以下情况会使索引失效：

- (1) `select * from t1 where c1 = nvl(c2, 0);`

- **索引列使用函数**：由于索引列 c1 后面的条件为函数，其无法选择索引扫描，而是选择全表扫描。
- 优化方法：应尽量避免在索引列中使用函数。

- (2) `select * from t1 where c1 + 1 = 10;`

- **索引列执行计算**：虽然 c1为索引列，但由于其进行索引列的计算，因此无法进行索引扫描。
- 优化方法：应尽量避免对索引列进行计算。

- (3) `select * from t1 where c1 = 6::numeric;` `select * from t1 where c3 = 6;`

- **索引列发生隐式类型转换**：第1条等号左侧的 c1 发生隐式类型转换，即转换为numeric类型后再与等号右侧的值进行比较，所以无法使用索引，最终语句走全表扫描；2条也一样，等号左侧为text类型，右边为int类型，此时将发生隐式转换，将c3列的text类型转换为int类型再与等号右边的6进行比较。
- 优化方法：将等号右侧的类型转换为与等号左侧类型相同的类型：`select * from t1 where c1 = 6;` `select * from t1 where c3 = 6::text;`
- 隐式类型转换导致的索引失效不只会出现在等值条件中，其他条件如：<,>,<>,like等条件中出现隐式类型转换都会导致索引失效。

索引失效 – 示例 (2)

- 以下情况会使索引失效：
 - (4) `select * from t1 where c1 = 10 or c2 = 20;`
 - **WHERE中的OR语句**：在 WHERE 子句中，如果在 OR 前的条件列是索引列，而在 OR 后的条件列不是索引列，那么索引会失效。如果OR条件前后为联合索引，且联合索引包含列的数目刚好为2个，也会导致索引失效。
 - (5) `select * from t1 where c3 like '%test%';`
 - **索引列使用模糊匹配条件等**：对索引列使用`like %xx`，`like xx%`和`like %xx%`会导致索引失效。

索引失效 – 其他场景

- 其他场景的索引失效：分区表建立GLOBAL索引，删除分区后导致索引失效。
 - （1）通常来说，如果创建分区表的索引时没有指定关键字LOCAL，其创建的索引为GLOBAL索引。对于分区表而言，如果建立的是GLOBAL索引，并且该分区表发生过删除分区的操作会导致索引变为UNUSABLE。如果后续我们不对该索引进行重建操作，该索引会一直处于失效状态。
 - （2）优化方法：
 - ① 创建分区键索引的时候使用局部索引，避免因为使用全局索引删除分区的时候失效。
 - ② 创建表之后尽量不要轻易改变表结构，避免索引失效的异常情况发生，删除分区的时候加上 UPDATE GLOBAL INDEX语句也可：

```
alter table index_part drop partition p1 update global index;
```
 - ③ 出现此情况也可以使用rebuild重建索引命令重建：

```
reindex index idx_part;
```

IndexOnlyScan回表简介

- IndexOnlyScan索引简介

- openGauss的indexOnlyScan操作是一种优化技术，它通过只使用索引来满足查询请求，而无需访问主表数据。这种操作可以大大提高查询性能，减少IO开销。

- IndexOnlyScan回表简介

- 有些情况下，IndexOnlyScan操作仍然需要回表。回表是指在索引中找到匹配的行后，还需要通过主表的物理地址获取完整的行数据。这是因为单独依靠索引树的信息并不能完成准确的查询，还是需要获取完整的行数据才能确定该数据是否有效。
- 需要注意的是，回表操作可能会增加额外的IO开销，因此在调优过程中，需要权衡使用IndexOnlyScan的场景和回表的开销，以获得最佳的查询性能。

IndexOnlyScan回表原因

- 为什么IndexOnlyScan需要回表？
 - (1) 索引列包含NULL值：如果索引列包含NULL值，那么在进行IndexOnlyScan时，需要回表来验证NULL值的存在。
 - (2) 索引列包含变长数据类型：如果索引列包含变长数据类型（如VARCHAR），那么在进行IndexOnlyScan时，需要回表来获取完整的变长数据。
 - (3) 在astore存储引擎中，如果该索引的数据发生修改操作，IndexOnlyScan为了判断该索引列数据的可见性，会先去查找VM，如果VM也无法判断该索引列数据的可见性，此时会回表访问完整的行的事务信息来确定该行的可见性。

IndexOnlyScan回表过多

- 如何判断IndexOnlyScan回表过多？
 - 利用EXPLAIN ANALYZE打印执行计划，通过查看IndexOnlyScan扫描算子下面的Heap Fetched字段可以判断是否回表过多。

```
openGauss=# explain analyze select c1 from index_lookup where c1 < 10000;
```

```
QUERY PLAN
```

```
-----  
Index Only Scan using idx_lookup on index_lookup (cost=0.00..262.12 rows=9920 width=4) (actual time=0.023..3.230  
rows=9999 loops=1)
```

```
  Index Cond: (c1 < 10000)
```

```
    Heap Fetches: 9999
```

```
  Total runtime: 4.150 ms  
(4 rows)
```

能波动大。

IndexOnlyScan回表过多 – 如何解决

- 如何解决回表过多问题？

- (1) IndexOnlyScan回表过多往往是由于表的存储方式采用astore导致需要回表，如果采用ustore可以避免回表问题。
- (2) IndexOnlyScan发生回表往往是因为在astore的存储模式下，出现大量更新、删除、插入操作导致行的可见性发生变化，单独访问VM(Visibility Map)无法解决可见性问题，需要读取表中实际行的事务信息才可以判断该行是否可见。此时可以通过VACUUM ANALYZE <my_table_name>;更新针对该表的VM信息，同时将所有行变为对当前事务可见。
- (3) 如果VACUUM ANALYZE开销太大，建议用Plan Hint的方式不走IndexOnlyScan，避免访问VM和回表带来的开销。
 - `select/*+ no indexonlyscan(index_lookup idx_lookup)*/ c1 from index_lookup where c1 < 10000;`

索引膨胀 (1)

- 为什么会出现索引膨胀？
 - 我们这里所说的索引膨胀都是针对openGauss的B-tree索引结构。
 - 对于B-tree索引结构，表中每个版本的行数据都对应一个所有行都可见且未死的索引条目。在执行VACUUM时，会清除B-tree索引中对应死元组的索引条目，但并不会回收索引占用的空间，而是把索引空间置为可复用的状态。如果新增索引数据符合条件，可以覆盖可复用的索引空间。
 - 随着这种操作的增多，同时没有符合条件的新增索引数据占据原有空间，会导致索引占用空间逐渐增大，最后造成索引膨胀的现象，这种现象在astore存储场景下比较常见。

索引膨胀 (2)

- 索引膨胀会造成什么性能问题？
 - 从某种程度上来说索引膨胀是不可避免的，一定程度的索引膨胀是正常的。但是如果索引过度膨胀会导致以下性能问题：
 - (1) 对于利用索引进行范围扫描而言，需要扫描更多的页面。
 - (2) openGauss会用shared buffers的内存空间去缓存索引页面，而随着索引膨胀也会导致对内存的使用增大，造成内存浪费。
 - (3) 索引膨胀会导致每个索引页面存储的索引条目更少，进而导致索引树的层级增高，降低索引查询效率。

索引膨胀 (3)

- 如何查看索引空间的大小？
 - 使用元命令\di+查看索引的空间大小：
 - \di+ my_index
- 如何解决索引膨胀？
 - (1) 通过重建索引解决索引膨胀
 - `reindex index my_index`
 - (2) 通过VACUUM FULL来解决索引膨胀
 - `vacuum full my_table`
 - (3) 联合索引中，如果对于大表建立过多的联合索引列，也会导致索引空间增大和索引性能问题，可以适当删除一些不必要的联合索引列，控制索引空间的大小，保证索引扫描的效率。

索引扫描死元组过多

- 1、什么是死元组？
 - 在openGauss的astore存储模式下，执行DELETE操作时，并不会从存储空间真正的将对应的行数据进行移除，仅仅是通过在行数据的头部的xmax字段进行删除标记。
 - 并且astore下的UPDATE操作会拆解成先INSERT新数据再DELETE旧数据的操作。这些被标记删除的元组也成为死元组（dead tuple）。
- 2、死元组对索引扫描造成什么影响？
 - 死元组增多会导致表空间膨胀，而普通的索引扫描需要回表查看数据。随着死元组的增多，索引扫描的回表次数也会增多，进而影响索引扫描性能。
- 3、如何解决死元组过多的问题？
 - 使用VACUUM操作可以回收死元组。
 - 建议使用VACUUM ANALYZE去进行该操作，因为VACUUM ANALYZE不止可以回收死元组还会更新优化器统计信息，获取更优的执行计划。

索引选择不合理 (1)

- 为什么会出现索引选择不合理现象？
 - 在openGauss中，是通过代价估算来进行底层扫描算子的选择，同时我们会默认表中的数据是按均匀分布的。
 - 在一些非均匀分布的场景下， openGauss会出现代价估算不准导致底层扫描算子选择错误的情况，索引选择不合理就是其中一种。
- 索引选择不合理一般分为两种场景：
 - （1）一张表上创建多个索引，代价预估不准导致索引选择不合理。
 - （2）两表关联中有多个索引，关联过程的代价估算不准导致索引选择不合理。

索引选择不合理 (2)

- 如何解决索引选择不合理现象？
 - 由于该现象较难排查，在正常场景下较难发现，容易误认为是索引效率较低的情况。
 - 一般来说对于定位索引选择不合理现象，需要明确知道应该选择哪个索引。该场景较常出现的情况为，同样的查询，同样的数据量，在A环境下效率高，在B环境下效率低。此时需要对比两个执行计划，查看是否有索引选择不合理的情况。
 - 通常来说，针对该现象只能通过Hint调优的方式去手动指定选择索引。

什么列才需要创建索引

- 索引作为数据库中一种快速查找数据的数据结构，其往往在查找数据量较少时性能较好，而在查找数据较多时性能不如全表扫描，同时索引还会引入一定的存储空间开销。因此在创建索引时，需要明确哪些字段适合创建索引，哪些字段适合直接进行全表扫描。通常来说，以下字段适合建立索引：
 - （1）where条件中选择性较强的字段，通常来说为选择率超过70%。
 - （2）关联结果集较小的关联字段。
 - （3）排序结果集较大的排序字段。
 - （4）分组结果集较大的分组字段。
 - （5）聚集结果较大的聚集字段。
- 在对于where条件较多的查询语句，可以对选择率前三的字段建立联合索引。联合索引不推荐包含字段过多，因为其在插入、更新、删除操作较多的场景下会导致索引快速膨胀，此时由于索引空间增大，会导致效率反而降低。
- 依据上述的规则来创建合理的索引，可以有效提升openGauss数据库的查询效率。

目录

1. 性能调优的总体思路与流程
2. 数据库整体性能调优
3. **SQL调优**
 - 相似算子选择及Plan Hint
 - 索引
 - 表结构优化
 - 优化器CBO
 - SQL性能诊断
 - 子查询调优和SQL语句改写

表结构优化原理 (1)

- 表结构优化是SQL调优的一个重要部分，主要涉及以下几个方面：
 - (1) **数据类型选择**：尽可能选择最适合存储的数据类型。
 - 例如，如果一个字段只存储0-255之间的数值，那么使用tinyint类型会比int类型更节省存储空间。
 - (2) **使用合适的索引**：索引可以大大提高查询速度。
 - 但是，过多的索引会导致插入、更新和删除操作变慢，因为每次数据变动都需要更新索引。因此，需要根据实际情况选择合适的索引。
 - (3) **规范化和反规范化**：规范化可以消除数据冗余，但可能会增加查询复杂性。
 - 反规范化可以减少查询复杂性，但可能会引入数据冗余。需要根据实际情况选择使用规范化还是反规范化。
 - (4) **分区和分表**：当表的数据量非常大时，可以考虑使用分区或分表来提高查询速度。
 - 分区是在物理上将表分成多个部分，每个部分都可以独立索引和查询。分表是将一个大表分成多个小表，每个小表都有自己的索引。
 - (5) **尽量避免使用外键**：外键可以保证数据的一致性和完整性，但外键约束会影响性能。

存储方式的选择

- 普通表优化场景
 - 对于普通表（非分区表）的优化可以从几个方面进行考虑，包括数据类型的选择（使用合理的存储方式），使用合适的索引以及对于规范化和反规范化的选择。
- 存储方式的选择
 - 进行数据库设计时，表设计上的一些关键项将严重影响后续整库的查询性能。表设计对数据存储也有影响：好的表设计能够减少I/O操作及最小化内存使用，进而提升查询性能。
 - 表的存储模型选择是表定义的第一步。客户业务属性是表的存储模型的决定性因素，依据下面表格选择适合当前业务的存储模型。

存储模型	适用场景
行存	点查询（返回记录少，基于索引的简单查询）。增删改比较多的场景。
列存	统计分析类查询（group、join多的场景）。

数据类型的选择

- (1) **尽量使用执行效率比较高的数据类型**

- 一般来说整型数据运算（包括=、>、<、≥、≤、≠等常规的比较运算，以及group by）的效率比字符串、浮点数要高。

- (2) **尽量使用短字段的数据类型**

- 长度较短的数据类型不仅可以减小数据文件的大小，提升IO性能；同时也可以减小相关计算时的内存消耗，提升计算性能。

- (3) **使用一致的数据类型**

- 表关联列尽量使用相同的数据类型。

- (4) 对于表中的numeric数据类型的字段，如果不追求特别高的精度尽量避免使用，openGauss中有关于numeric类型运算的效率比float或double precision低很多。

使用分区表

- 有时我们需要操作的表非常大以至于无法全部都放在内存中，或者只在表的最后部分有热点数据，其他均是历史数据，那么我们可以考虑使用分区表。
- 1、什么是分区表？
 - 分区表是指根据一定规则，将数据库中的一张表分解成多个更小的，容易管理的部分。从逻辑上看，只有一张表，但是底层却是由多个物理分区组成。
- 2、分区表有什么优点？
 - （1）改善查询性能：对分区对象的查询可以仅搜索自己关心的分区，提高检索效率。
 - （2）增强可用性：如果分区表的某个分区出现故障，表在其他分区的数据仍然可用。
 - （3）方便维护：如果分区表的某个分区出现故障，需要修复数据，只修复该分区即可。

分区表分类

- openGauss数据库支持的分区表为一级分区表和二级分区表，其中一级分区表包括范围分区表、间隔分区表、列表分区表、哈希分区表四种，二级分区表包括范围分区、列表分区、哈希分区两两组合的九种。
 - （1）**范围分区表**：将数据基于范围映射到每一个分区。这个范围是由创建分区表时指定的分区键决定的。这种分区方式是最为常用的，并且分区键经常采用日期，例如将销售数据按照月份进行分区。
 - （2）**间隔分区表**：特殊的范围分区表，相比范围分区表，新增间隔值定义，可以根据间隔值自动创建分区。
 - （3）**列表分区表**：将数据中包含的键值分别存储在不同的分区中，依次将数据映射到每一个分区，分区中包含的键值由创建分区表时指定。
 - （4）**哈希分区表**：将数据根据内部哈希算法依次映射到每一个分区中。
 - （5）**二级分区表**：由范围分区、列表分区、哈希分区任意组合得到的分区表，其一级分区和二级分区均可以使用前面三种定义方式。

分区表的LOCAL索引和GLOBAL索引 (1)

- (1) 全局索引
 - 全局索引(Global Index)是针对整个分区表的索引。
 - `create index on my_part_tbl(c1) global;`
- (2) 本地索引
 - 本地索引(Local Index)是针对分区表中单个分区的索引。
 - `create index on my_part_tbl(c1) local;`
- (3) 在查询时，如果涉及跨分区则无法使用本地索引，只能使用全局索引。
- (4) 在性能上，频繁的插入和更新对于全局索引的维护开销也会更大。

使用局部聚簇

- 局部聚簇（Partial Cluster Key）是列存下的一种技术。这种技术可以通过min/max稀疏索引较快的实现基表扫描的filter过滤。Partial Cluster Key可以指定多列，但是一般不建议超过2列。
- Partial Cluster Key的选取原则：
 - 受基表中的简单表达式约束。这种约束一般形如col op const，其中col为列名，op为操作符 =、>、>=、<=、<，const为常量值。
 - 尽量采用选择度比较高（过滤掉更多数据）的简单表达式中的列。
 - 尽量把选择度比较低的约束col放在Partial Cluster Key中的前面。
 - 尽量把枚举类型的列放在Partial Cluster Key中的前面。

目录

1. 性能调优的总体思路与流程
2. 数据库整体性能调优
3. **SQL调优**
 - 相似算子选择及Plan Hint
 - 索引
 - 表结构优化
 - 优化器CBO
 - SQL性能诊断
 - 子查询调优和SQL语句改写

优化器介绍

- 优化器目标：通过代价估算在备选执行计划中选择实际执行时最优的计划，基于规则的查询重写（逻辑优化）和基于代价的查询优化（物理优化）。
- RBO(rule based optimizer)，基于规则的优化（逻辑优化）。
 - 从关系代数的角度进行查询语句的等价变换，从逻辑角度进行查询的简化。
 - 传统意义上，查询重写都是基于规则的，但随着SQL语言的复杂和查询重写规则的多样性，基于代价的查询重写技术出现的。
- CBO(cost based optimizer)，基于代价的优化（物理优化）。
 - 对于逻辑优化输出的最简查询，罗列各种可行的计算计划，比较它们的代价并给出最优计划。
 - 通常情况下计划的可能性随表个数的增多呈指数级上升，所以罗列所有可能性是不现实的。
 - 不同的执行计划，执行时间可能差上千倍，所以物理优化的选择对查询性能来说很关键。

优化器CBO介绍

- openGauss是基于代价估算生成的最优执行计划。
- CBO(Cost-Based Optimization)优化器根据数据的统计信息、基数估计和算子代价模型，估算出执行所需要的代价，最终选择代价最小的执行计划作为SQL的具体执行方式。
 - 数据的统计信息一般包括表的行数、字段宽度、NULL记录比率、distinct值、MCV值等特征值。
 - 基数估计会估算各个算子中间结果的行数等信息，主要包括两个部分：带过滤条件的行数估算和JOIN结果的行数估算，而这两部分都基于正确的选择率得出。

优化器CBO – 常见的统计信息问题

- 在openGauss数据库中，统计信息是CBO生成执行计划的源数据。
- 没有收集统计信息或者统计信息陈旧往往会造成执行计划严重劣化，从而导致性能问题。
- 常见的统计信息问题如下：
 - (1) rows预估值高于实际值。
 - 主要场景一般为少量数据插入空表，此时统计信息会错误估计，结果偏高，但是通过修正后并不能优化执行计划。
 - (2) 预估rows与实际rows有偏差但不多。
 - 这种场景大多体现在集中式中，由于集中式统计信息的使用存在动态扫表，表数据存储并非十分均匀，可能存在相对偏差，同理，可以通过添加HINT的方式判定是否为统计信息不准导致。
 - (3) Plan Hint影响对执行计划的选择。
 - 部分SQL通过添加HINT修正统计信息，改变了执行计划，会误解为算子选择错误导致，实际需要判定，是否数据量级会跟执行计划选择有关。
 - 这种场景可通过尝试制造不同量级数据，并通过添加HINT保证统计信息的准确性，判定问题根因是否与统计信息有关。

优化器CBO – 统计信息问题识别步骤

- 判断SQL语句是否有统计信息问题的大致步骤如下：
 - (1) 执行EXPLAIN ANALYZE语句，查看预估rows与实际rows是否相差过大。如果相差过大，则可能是统计信息的问题。
 - (2) 将执行语句改写为SELECT COUNT(*) FROM xxx，进一步确认第一步的判断是否正确。
 - (3) 查询对应系统表检测统计信息是否及时收集或更新。如果没有，可尝试通过手动更新统计信息调优。
 - (4) 如果第三步发现统计信息正确，或者第三步收集后执行效率并未改善，可尝试通过添加HINT的方式，观测执行计划是否被优化，时延是否降低。如果有明显的改善，则该问题大概率是统计信息问题导致的。

优化器CBO – 更新统计信息

- 统计信息问题的基本判断和修正手段有两种：添加HINT和手动更新统计信息。
 - 手动收集统计信息一半适用于一些频繁使用的表，添加几行SQL就可能提升整个脚本的性能，但是需要考虑添加SQL后是否造成性能劣化。如循环中添加ANALYZE语句，可能会导致大量时延损耗，整体性能不升反降。
 - 添加HINT方式适用于复杂SQL中，属于比较简单，副作用较小的方式，但是在脚本中统计信息异常的执行计划偏多，或者基表增删数据数量较大或频率较高等场景，添加HINT的方式不一定能解决问题。

- 收到更新统计信息：

- （1）使用以下命令更新某个表或者整个database的统计信息。

```
ANALYZE tablename;  
ANALYZE;
```

- （2）使用以下命令进行多列统计信息相关操作。

```
ANALYZE tablename ((column_1, column_2));  
ALTER TABLE tablename ADD STATISTICS ((column_1, column_2));  
ALTER TABLE tablename DELETE STATISTICS ((column_1, column_2));
```

CBO相关参数调优 (1)

GUC参数	参数说明
var_eq_const_selectivity	控制优化器是否使用直方图计算整型常量的选择率。
partition_page_estimation	控制分区表页面是否通过剪枝结果进行页面估算优化，只包括分区表和local索引页面，不包括全局索引页面。估算公式为： 估算后页面 = 分区表总页面 * (剪枝后分区数 / 总分区数)。
partition_iterator_elimination	控制分区表在分区剪枝结果为一个分区时，是否通过消除分区迭代算子来提升执行效率。
enable_functional_dependency	控制函数依赖统计信息的使用。 设置为“on”，开启两个功能：执行ANALYZE生成的多列统计信息包含函数依赖统计信息。计算选择率会使用函数依赖统计信息。 设置为“off”，此两个功能不生效：执行ANALYZE生成的多列统计信息不包含函数依赖统计信息。计算选择率不会使用函数依赖统计信息。

CBO相关参数调优 (2)

GUC参数	参数说明
seq_page_cost	设置优化器计算一次顺序磁盘页面抓取的开销。 默认值是1
random_page_cost	设置优化器计算一次非顺序抓取磁盘页面的开销。 默认值是4
cpu_tuple_cost	设置优化器计算在一次查询中处理每一行数据的开销。 默认值是0.01
cpu_index_tuple_cost	设置优化器计算在一次索引扫描中处理每条索引的开销。 默认值是0.005

CBO优化器案例解析 (1)

- 为多个可行的物理执行代价评分（估算），选择最优的计划，输出到执行器，其关注点：
 - 结果集行数。
 - 表示优化器进行估算时当前层结果集的估算行数。
 - 执行代价估算。
 - 由两个数表示，第一个数为输出第一条元组的估算代价，第二个数为输出所有元组的估算总代价。总代价包含所有子算子的总代价。
 - join路径搜索与选择。
 - 标识执行器使用哪个算子来执行当前层计划

```
openGauss=# explain select * from t1 join t2 on t1.a=t2.a;
               QUERY PLAN
-----
Hash Join (cost=270.00..552.50 rows=10000 width=16)
  Hash Cond: (t1.a = t2.a)
    -> Seq Scan on t1 (cost=0.00..145.00 rows=10000 width=8)
    -> Hash (cost=145.00..145.00 rows=10000 width=8)
          -> Seq Scan on t2 (cost=0.00..145.00 rows=10000 width=8)
(5 rows)
```

CBO优化器案例解析 (2)

- 行数估算。
 - 目的：估算连接过程中结果集大小，为代价估算做准备。
- 通过analyze命令，收集基表统计信息，包括：
 - 各数据表的规模信息，包括行数，页面数。
 - 表中各列的统计信息，包括distinct值，MCV和直方图。
 - distinct值，用于记录该列不同值的个数。
 - NULL值的比例。
 - MCV(most common value)，用于记录倾斜信息。
 - 直方图，用于记录数据分布情况。

目录

1. 性能调优的总体思路与流程
2. 数据库整体性能调优
3. **SQL调优**
 - 相似算子选择及Plan Hint
 - 索引
 - 表结构优化
 - 优化器CBO
 - SQL性能诊断
 - 子查询调优和SQL语句改写

WDR报告介绍

- WDR报告是openGauss提供的一种性能收集和分析工具，通过此报告可以了解数据库整体运行情况。
 - 基于WDR Snapshot数据表汇总、统计，生成性能报告。
 - 默认初始化用户或监控管理员用户可以生成报告。
- WDR Snapshot用于定时查询系统性能视图并生成快照：
 - （1）通过guc参数enable_wdr_snapshot开启，默认关闭。
 - （2）仅采集schema为dbms_perf下的性能视图。
 - （3）采集周期默认为60分钟，通过guc参数wdr_snapshot_interval调整。
 - （4）保存时间默认为1天，通过wdr_snapshot_retention_days调整。
 - （5）可通过调用系统函数create_wdr_snapshot手动生成快照。

生成WDR报告

- 数据库中只有一个快照或者需要查看在当前时间段数据库的监控数据，可以选择手动执行快照操作，该命令需要用户具有sysadmin权限。
 - (1) 执行如下命令查询已经生成的快照，以获取快照的snapshot_id。
 - `select * from snapshot.snapshot;`
 - (2) (可选) 执行如下命令手动创建快照。
 - `select create_wdr_snapshot();`
 - (3) 执行如下步骤生成性能报告。
 - a. 执行如下命令生成格式化性能报告文件。
 - `\a \t \o <file_path>`
 - b. 执行如下命令将查询到的信息写入性能报告中。
 - `select generate_wdr_report(begin_snap_id bigint, end_snap_id bigint, report_type cstring, report_scope cstring, node_name cstring);`

生成WDR报告 - generate_wdr_report 函数参数说明

● generate_wdr_report 函数参数说明：

- begin_snap_id：查询时间段开始的snapshot的id（表snapshot.snaoshot中的snapshot_id）。
- end_snap_id：查询时间段结束snapshot的id。默认end_snap_id大于begin_snap_id（表snapshot.snaoshot中的snapshot_id）。
- report_type：指定生成report的类型。例如：summary：汇总数据；detail：明细数据；all：包含summary和detail。
- report_scope：指定生成report的范围，可以为cluster或者node。
- cluster：数据库级别的信息。
- node：节点级别的信息。
- node_name：在report_scope指定为node时，需要把该参数指定为对应节点的名称。（节点名称可以执行select * from pg_node_env;查询）；在report_scope为cluster时，该值可以省略或者指定为空或NULL。

查看WDR报告

- Summary:

- (1) 最耗时等待事件、最耗时等待事件的种类。
- (2) CPU、IO、Memory统计信息。

- Detail:

- (1) 时间模型、SQL统计信息、SQL具体信息、系统等待事件。
- (2) Cache/IO统计、应用统计、对象 (table、index、bad block) 统计。
- (3) 系统配置。

慢SQL介绍

- 慢SQL能根据用户提供的执行时间阈值(log_min_duration_statement)，记录所有超过阈值的执行完毕的作业信息。
- 慢SQL提供表和函数两种维度的查询接口，用户从接口中能查询到作业的执行计划，开始、结束执行时间，执行查询的语句，行活动，内核时间，CPU时间，执行时间，解析时间，编译时间，查询重写时间，计划生成时间，网络时间，IO时间，网络开销，锁开销等。
- openGauss对慢SQL功能进行了安全增强，所有信息都是脱敏的。

查询慢SQL

- (1) 执行命令查看数据库实例中SQL语句执行信息。
 - `select * from dbe_perf.get_global_full_sql_by_timestamp(start_timestamp, end_timestamp);`
- (2) 执行命令查看数据库实例中慢SQL语句执行信息。
 - `select * from dbe_perf.get_global_slow_sql_by_timestamp(start_timestamp, end_timestamp);`
- (3) 查看当前节点SQL语句执行信息。
 - `select * from statement_history;`

慢SQL特性约束

- (1) 目前的SQL跟踪信息，基于正常的执行逻辑。执行失败的SQL，其跟踪信息不具有准确的参考价值。
- (2) 节点重启，可能导致该节点的数据丢失。
- (3) SQL语句执行完立即退出会话，可能会丢失该会话未刷新到系统表中的数据。
- (4) 通过GUC参数设置收集SQL语句的数量，如果超过阈值，新的SQL语句执行信息不会被收集。
- (5) 通过GUC参数设置单条SQL语句收集的锁事件详细信息的最大字节数，如果超过阈值，新的锁事件详细信息不会被收集。
- (6) 通过异步刷新方式刷新用户执行中的SQL信息，所以用户Query执行结束后，存在查询相关视图函数结果暂时延。
- (7) 部分指标信息（行活动、Cache/IO、时间分布等）依赖于dbe_perf.statement视图收集，如果该视图对应记录数超过预定大小（依赖GUC:instr_unique_sql_count），则本特性可能不收集相关指标。
- (8) statement_history表相关函数以及视图中details字段为二进制格式，如果需要解析详细内容，请使用对应函数：`pg_catalog.statement_detail_decode(details, 'plaintext', true)`。

目录

1. 性能调优的总体思路与流程
2. 数据库整体性能调优
3. **SQL调优**
 - 相似算子选择及Plan Hint
 - 索引
 - 表结构优化
 - 优化器CBO
 - SQL性能诊断
 - 子查询调优和SQL语句改写

子查询和子链接介绍

- 什么是子查询和子链接？
 - 应用程序通过SQL语句来操作数据库时会使用大量的子查询，这种写法比直接对两个表做连接操作在结构上和思路更清晰，尤其是在一些比较复杂的查询语句中，子查询有更完整、更独立的语义，会使SQL对业务逻辑的表达更清晰更容易理解，因此得到了广泛的应用。
 - openGauss根据子查询在SQL语句中的位置把子查询分成了子查询、子链接两种形式。
 - (1) **子查询 SubQuery**:
 - 对应于查询解析树中的范围表RangeTblEntry，更通俗一些指的是出现在FROM语句后面的独立的SELECT语句。
 - (2) **子链接 SubLink**:
 - 对应于查询解析树中的表达式，更通俗一些指的是出现在where/on子句、targetlist里面的语句。
 - (3) SubQuery的本质是范围表，而SubLink的本质是表达式。

子查询类型 (1)

- 子查询从大类上来看，分为非相关子查询和相关子查询。
 - (1) 非相关子查询None-Related SubQuery
 - 子查询的执行不依赖于外层父查询的任何属性值。这样子查询具有独立性，可独自求解，形成一个子查询计划先于外层的查询求解。
 - `select t1.c1, t1.c2 from t1 where t1.c1 in ( select c2 from t2 where t2.c2 in (2,3,4) );`
 - (2) 相关子查询Correlated-SubQuery
 - 子查询的执行依赖于外层父查询的一些属性值，如下列示例`t2.c1 = t1.c1`条件中的`t1.c1`，作为内层查询的一个条件。这样的子查询不具备独立性，需要和外层查询按分组进行求解。
 - 相关子查询的特别之处在于，其本身是不完整的：它的闭包中包含一些外层查询提供的参数。
 - `select t1.c1,t1.c2 from t1 where t1.c1 in ( select c2 from t2 where t2.c1 = t1.c1 AND t2.c2 in (2,3,4) );`

子查询类型 (2)

- 根据产生的数据来分类，子查询可以分成以下几种：
 - 1. 标量子查询：
 - 输出一个只有一行一列的结果表，这个标量值就是它的结果。如果结果为空（0行），则输出一个NULL。但是注意，超过1行结果是不被允许的，会产生一个运行时异常。
 - 标量子查询可以出现在任意包含标量的地方，例如 SELECT、WHERE 等子句里。
 - 例1：出现在 WHERE 子句中的标量子查询：`select t1.a from t1 where 100 < (select sum(b) from t2 where t2.c = t1.c);`
 - 例2：出现在 SELECT 子句中的标量子查询：`select t1.a ,(select t2.b from t2 where t1.a = t2.a) as b from t1;`
 - 2. 存在性检测子查询：
 - 特指 EXISTS 的子查询，返回一个布尔值。如果出现在 WHERE 中，这就是我们熟悉的 Semi-Join。当然，它可能出现在任何可以放布尔值的地方。
 - 例：`select t1.a from t1 where t1.b = 8 and EXISTS (select * from t2 where t1.c = t2.c);`

子查询类型 (3)

- 3. 集合比较子查询

- 特指IN、SOME、ANY的查询，返回一个布尔值，它可能出现在任何可以放布尔值的地方。常用的形式有：x = SOME(Q)（等价于 x IN Q）或 X <> ALL(Q)（等价于 x NOT IN Q）。
- 例：select t1.a from t1 where t1.b <> all (select t2.b from t2);
- 以上实例是一个集合比较的非相关子查询。

子查询调优 (1)

- 案例1：改写SQL语句，消除相关子查询。

```
SELECT  t1.name,  
        (SELECT t2.name FROM t2 WHERE t2.id = t1.id) as department_name  
FROM    t1;
```

- 为提高查询性能，将上述SQL语句中的子查询消除，改写为：

```
select  
    t1.name, t2.name as department_name  
from t1 left join t2  
on t1.id = t2.id;
```

- 使用JOIN通常比子查询更高效，因为数据库引擎可以更好地优化JOIN操作。JOIN操作通常在查询开始时执行，将相关表的数据放在一起，然后进行过滤。而子查询可能会在查询的每个步骤中都执行一次，特别是相关联的外部查询列越多、数据量越大时，这种差异就越明显。然而，应该注意的是，并不是所有情况下JOIN都比子查询更优，所以每次优化都需要根据实际情况和查询执行计划来判断。

子查询调优 (2)

- 案例2：改写SQL语句，优化存在性检测子查询。

```
SELECT  t1.name, t1.addr  
FROM t1  
WHERE EXISTS ( SELECT 1 FROM t2 WHERE t2.id = t1.id AND t2.location = 'abc');
```

- 为提高查询性能，将上述SQL语句改写为：

```
SELECT  t1.name, t1.addr  
FROM t1  
INNER JOIN T2  
ON t1.id = t2.id  
WHERE t2.location = 'abc';
```

子查询调优 (3)

- 案例3：改写SQL语句，优化集合比较子查询。

```
SELECT t1.name, t1.addr  
FROM t1  
WHERE t1.id IN ( SELECT t2.id FROM t2 WHERE t2.location = 'abc');
```

- 为提高查询性能，将上述SQL语句改写为：

```
SELECT t1.name, t1.addr  
FROM t1  
INNER JOIN t2  
ON t1.id = t2.id  
WHERE t2.location = 'abc';
```

数据上都执行子查询。

子链接SubLink分类

- 子链接SubLink有哪些分类？

- (1) exist_sublink: 对应EXIST、NOT EXIST语句。
- (2) any_sublink: 对应op ALL(select...)语句，其中OP可以是IN、<、>、=操作符。
- (3) all_sublink: 对应op ALL(select...)语句，其中OP可以是IN、<、>、=操作符。
- (4) rowcompare_sublink: 对应record op (select ...)语句。
- (5) expr_sublink: 对应(SELECT with single targetlist item ...)语句。
- (6) array_sublink: 对应ARRAY(select...)语句。
- (7) cte_sublink: 对应with query(...)语句。

子链接SubLink的优化 (1)

- openGauss针对SubLink的优化策略主要是让内层的子查询提升（pullup），能够和外表直接做关联查询，从而避免生成SubPlan+Broadcast内表的执行计划。
- 判断子查询是否存在性能风险，可以通过explain查询语句查看Sublink的部分是否被转换成SubPlan的执行计划。
- 目前openGauss支持的Sublink-Release场景：
 - （1）IN-Sublink无相关条件：不能包含上一层查询的表中的列，不能包含易变函数。
 - （2）Exist-Sublink包含相关条件：Where子句中必须包含上一层查询的表中的列，子查询的其它部分不能含有上层查询的表中的列。其它限制包括：
 - 子查询必须有from子句。
 - 子查询不能含有with子句。
 - 子查询不能含有聚集函数。
 - 子查询里不能包含集合操作、排序、limit、windowagg、having操作。
 - 不能包含易变函数。

子链接SubLink的优化 (2)

- (3) 包含聚集函数的等值相关子查询的提升。
 - 子查询的where条件中必须含有来自上一层的列，而且此列必须和子查询本层涉及表中的列做相等判断，且这些条件必须用and连接。其它地方不能包含上层的列。其它限制条件如下：
 - 子查询中where条件包含的表达式(列名)必须是表中的列。
 - 子查询的Select关键字后，必须有且仅有一个输出列，此输出列必须是聚集函数(如max)，并且聚集函数的参数(t2.c2)不能是来自外层表(t1)中的列。聚集函数不能是count。
 - 子查询必须是from子句。
 - 子查询中不能有groupby、having、集合操作。
 - 子查询只能是inner join。
 - 子查询的targetlist中不能包含返回set的函数。
 - 子查询的where条件中必须含有来自上一层的列，而且此列必须和子查询层涉及表中的列做相等判断，且这些条件必须用and连接。其它地方不能包含上层的上层中的列。

子链接SubLink的优化 (4)

- (4) 提升OR子句中的SubLink。例如下列SQL语句：

```
select a, c from t1 where t1.a = (select avg(a) from t3 where t1.b = t3.b) or exists (select * from t4 where t1.c = t4.c);
```

- 当WHERE过滤条件中有OR连接的EXIST相关SubLink，将OR-ed连接的EXIST相关子查询OR字句的提升过程：

- 1. 提取where条件中，or子句中的opExpr为：t1.a = (select avg(a) from t3 where t1.b = t3.b)
- 2. 这个op操作中包含subquery，判断是否可以提升，如果可以提升，重写subquery为：select avg(a), t3.b from t3 group by t3.b，生成not null条件t3.b is not null，并将这个opexpr用这个not null条件替换。此时SQL

```
select a, c from t1 left join (select avg(a) avg, t3.b from t3 group by t3.b) as t3 on (t1.a = avg and t1.b = t3.b)
where t3.b is not null or exists (select * from t4 where t1.c = t4.c);
```

- 3. 再次提取or子句中的exists sublink，exists (select * from t4 where t1.c = t4.c)，判断是否可以提升，如果可以提升，转换subquery为：select t4.c from t4 group by t4.c生成NotNull条件t4.c is not null提升查询，
- ```
select t1.a, t1.c from t1 left join (select avg(a) avg, t3.b from t3 group by t3.b) as t3 on (t1.a = avg and t1.b = t3.b) left join (select t5.c
from t5 group by t5.c) as t5 on (t1.c = t5.c) where t3.b is not null or t5.c is not null;
```

# 子链接SubLink的优化 (5)

- openGauss不支持的Sublink-Release场景：

- 除了以上场景之外都不支持Sublink提升，因此关联子查询会被计划成SubPlan+Broadcast的执行计划，当inner表的数据量较大时则会产生性能风险。
- 如果相关子查询中跟外层的两张表做join，那么无法提升该子查询，需要通过将父SQL创建成with子句，然后再跟子查询中的表做相关子查询查询。例如：

```
select distinct t1.a, t2.a
from t1 left join t2 on t1.a=t2.a and not exists (select a,b from test1 where test1.a=t1.a and test1.b=t2.a);
```

- 改写为：

```
with temp as
(
 select * from (select t1.a as a, t2.a as b from t1 left join t2 on t1.a=t2.a)
)
select distinct a,b
from temp
where not exists (select a,b from test1 where temp.a=test1.a and temp.b=test1.b);
```

# SQL改写调优 (1)

- 根据数据库的SQL执行机制以及大量的实践表明，通过一定的规则调整SQL语句，在保证结果正确的基础上，能够提高SQL执行效率。如果遵守这些规则，常常能够大幅度提升业务查询效率。
- (1) 使用union all代替union。
  - union在合并两个集合时会执行去重操作，而union all则直接将两个结果集合并、不执行去重。如果通过业务逻辑已确认两个集合不存在重叠，可用union all替代union以便提升性能。
- (2) join列增加非空过滤条件。
  - 若join列上的NULL值较多，则可以加上is not null过滤条件，以实现数据的提前过滤，提高join效率。
- (3) not in转not exists。
  - not in语句需要使用nestloop anti join来实现，而not exists则可以通过hash anti-join来实现。在join列不存在null值的情况下，not exists和not in等价。
- (4) 选择hashagg。
  - 查询中GROUP BY语句如果生成了groupagg+sort的plan性能会比较差，可以通过加大work\_mem的方法生成hashagg的plan，因为不用排序而提高性能。

# SQL改写调优 (2)

- (5) 尝试将函数替换为case语句。
  - case语句比调用函数性能好，可以根据情况把可下推函数的函数改成CASE表达式。
- (6) 避免对索引使用函数或表达式运算。
  - 对索引使用函数或表达式运算会停止使用索引转而执行全表扫描。
- (7) 尽量避免在where子句中使用!=或<>操作符、null值判断、or连接、参数隐式转换。
- (8) 对复杂SQL语句进行拆分。
  - 对于过于复杂并且不易通过以上方法调整性能的SQL可以考虑拆分的方法，把SQL中某一部分拆分成独立的SQL并把执行结果存入临时表，拆分常见的场景包括但不限于：
    - 作业中多个SQL有同样的子查询，并且子查询数据量较大。
    - Plan cost计算不准，导致子查询hash bucket太小，比如实际数据1000W行，hash bucket只有1000。
    - 函数（如substr、to\_number）导致大数据量子查询选择度计算不准。